

UNIVERSIDADE DE BRASÍLIA – UnB
FACULDADE DE TECNOLOGIA (FT)
DEPARTAMENTO DA ENG. DE PRODUÇÃO

Disciplina: Sistemas de Suporte à Decisão

Professor: Prof. Dr. André Serrano

Planejamento de Manutenção Preditiva na Indústria 4.0:

Um MVP de Apoio à Decisão Baseado em
Aprendizado de Máquina sob Desbalanceamento
Severo

Discente: Fernando Cruvinel Casas

Matrícula: 221006511

Repositório: [GitHub/SSD_T2.ipynb](https://github.com/FernandoCasas/SSD_T2.ipynb)

Brasília, DF

2026

Resumo

Este trabalho apresenta o desenvolvimento de um Produto Mínimo Viável (MVP) voltado ao suporte à tomada de decisão para a gestão de manutenção preditiva em equipamentos industriais. Utilizando o conjunto de dados *AI4I 2020 Predictive Maintenance Dataset* (Kaggle), composto por 10.000 instâncias de telemetria operacional com atributos mecânicos e térmicos, o estudo estrutura o processo decisório em três eixos analíticos: Risco de Falha, Degradação da Ferramenta e Estresse Operacional. O desafio central reside no desbalanceamento severo da variável alvo (*Target*), na qual as falhas representam apenas 3,5% das observações. A metodologia implementa um pipeline reproduzível de pré-processamento via Scikit-Learn com padronização escalar, codificação categórica e partição estratificada de dados. Foram treinados e comparados três modelos sob balanceamento de classes: *Dummy Baseline*, Regressão Logística e *Random Forest*. Os resultados demonstraram que métricas clássicas baseadas em acurácia (96,50%) são enganosas no contexto de falhas raras, enquanto a Regressão Logística balanceada priorizou a capacidade de captura de anomalias (Recall de 38,57%). A extração de importância dos atributos comprovou que torque, velocidade rotacional e desgaste acumulado são os vetores determinantes de parada. Por fim, formula-se uma camada prescritiva detalhando regras de acionamento em sistemas SCADA/IoT, trade-offs econômicos entre falsos positivos e falsos negativos, e a automação de ordens de serviço.

Palavras-chave: Sistemas de Suporte à Decisão; Manutenção Preditiva; Aprendizado de Máquina; Dados Desbalanceados; Indústria 4.0; Florestas Aleatórias.

Sumário

1	Introdução	3
1.1	O Problema de Decisão	3
1.2	Hipótese e Objetivos	4
1.3	A Base de Dados: AI4I 2020 Predictive Maintenance	4
2	Método	5
2.1	Configuração do Ambiente e Reprodutibilidade	5
2.2	Carga e Validação dos Dados	6
2.3	Limpeza e Prevenção de Vazamento de Dados	8
2.4	Análise Exploratória de Correlações	8
2.5	Engenharia de Atributos e Pipeline de Transformação	9
2.6	Modelagem Preditiva e Balanceamento de Classes	10
2.7	Matriz de Confusão	11
2.8	Camada Prescritiva e Importância dos Atributos	12
3	Resultado	14
3.1	Auditoria e Sanidade da Base	14
3.2	Análise Exploratória e Dinâmica dos Sensores	14
3.3	Desempenho Comparativo dos Modelos	15
3.4	Diagnóstico do Trade-Off e a Ilusão da Acurácia	15
3.5	Avaliação pela Matriz de Confusão	16
3.6	Importância Relativa dos Atributos	17
3.7	Camada Prescritiva: Diretrizes Gerenciais de Intervenção	18
3.7.1	1. Parametrização de Alarmes no Sistema SCADA/IoT	18
3.7.2	2. Otimização Econômica do Trade-Off de Erros	18
3.7.3	3. Automação de Ordens de Serviço Preventivas	18
4	Conclusão	19
4.1	Limitações do Estudo	20
4.2	Trabalhos Futuros	20

1 Introdução

A transformação digital no contexto da Indústria 4.0 reposicionou o papel da manutenção nas plantas fabris contemporâneas. Historicamente tratada como um centro de custo reativo — operando sob o paradigma corretivo (*run-to-failure*) ou preventivo baseado em tempo decorrido —, a gestão de ativos industriais demanda hoje estratégias preditivas orientadas por dados. A ocorrência de paradas não programadas em linhas de manufatura de alta cadência gera prejuízos multifacetados, incluindo lucros cessantes, quebra em cascata de subcomponentes mecânicos, penalidades por atraso na entrega de pedidos e riscos severos à integridade dos operadores.

O avanço na instrumentação de máquinas-ferramenta por meio de redes de sensores de Internet das Coisas Industrial (*IIoT*) viabilizou o registro em fluxo contínuo de variáveis de estado operacional, tais como gradientes de temperatura, rotação angular, torque e tempo acumulado de desgaste da ferramenta de corte [source: 2]. Todavia, a mera coleta de grandes volumes de telemetria não assegura eficiência operacional se não houver um Sistema de Suporte à Decisão (SSD) estruturado, capaz de transformar sinais brutos de sensores em probabilidades calibradas de falha e planos acionáveis de intervenção.

1.1 O Problema de Decisão

O problema gerencial enfrentado pela organização consiste em mitigar o risco de falhas catastróficas em centros de usinagem sem incorrer no excesso de paradas preventivas prematuras, que descartam ferramental com vida útil remanescente [source: 2]. Para subsidiar a equipe de engenharia e gestão de operações antes de despachar equipes de campo, o sistema de suporte à decisão deve responder de forma objetiva a três eixos centrais de incerteza, conforme sintetizado na Tabela 1 [source: 2].

Tabela 1: Eixos de Decisão do Planejamento de Manutenção Preditiva [source: 2].

Eixo	Pergunta Gerencial de Decisão	Variável na Base
RISCO	Qual é a probabilidade da máquina falhar nas próximas horas dada a telemetria atual?	Target (0 = Normal, 1 = Falha)
DESGASTE	Como o tempo de contato e desgaste cumulativo aproxima a ferramenta do colapso?	Tool wear [min]
OPERAÇÃO	Qual é o impacto do estresse cinemático e mecânico imediato sobre o equipamento?	Torque [Nm], Rotational speed [rpm]

A tomada de decisão gerencial atua diretamente sobre um conjunto de variáveis físicas e de especificação monitoradas continuamente [source: 2]:

1. **Tipo de Equipamento (Type):** Variante dimensional e de qualidade do produto (*Low* – L, *Medium* – M, *High* – H), definindo a robustez do maquinário e tolerância a sobrecargas [source: 2].
2. **Condições Térmicas:** Temperatura ambiente do ar (*Air temperature* [K]) e temperatura gerada na zona de usinagem (*Process temperature* [K]) [source: 2].
3. **Dinâmica Cinemática:** Velocidade angular do fuso (*Rotational speed* [rpm]) e esforço de torção exigido pelo corte (*Torque* [Nm]) [source: 2].
4. **Degradação Mecânica:** Minutos acumulados de trabalho contínuo sem afiação da ferramenta (*Tool wear* [min]) [source: 2].

1.2 Hipótese e Objetivos

A hipótese norteadora deste projeto postula que: *os sinais combinados de estresse cinemático (torque e rotação) aliados ao desgaste acumulado da ferramenta carregam assinaturas estatísticas suficientes para antecipar falhas de equipamento antes que ocorra a quebra física.*

Para validar a hipótese, definem-se os seguintes objetivos específicos [source: 2]:

- Auditar a integridade da base de dados industrial, removendo identificadores e prevenindo vazamento de informação (*data leakage*) [source: 2];
- Estruturar um pipeline automatizado de aprendizado de máquina capaz de operar sob o desbalanceamento de classes inerente a falhas raras [source: 2];
- Treinar e comparar modelos estatísticos supervisionados (*Dummy Baseline*, Regressão Logística e Florestas Aleatórias) [source: 2];
- Diagnosticar o trade-off crítico entre precisão e revocação (Falsos Positivos vs. Falsos Negativos) no contexto industrial [source: 2];
- Desenvolver uma camada prescritiva de regras de negócio acionáveis no sistema supervisório fabril [source: 2].

1.3 A Base de Dados: AI4I 2020 Predictive Maintenance

Utilizou-se o conjunto de dados *AI4I 2020 Predictive Maintenance Dataset*, disponibilizado publicamente no repositório Kaggle e no *UCI Machine Learning Repository*. O

dataset reflete uma simulação realista de eventos de fresagem sintéticos baseados em parâmetros reais de usinagem [source: 2].

A base conta com 10.000 instâncias e parâmetros ambientais e mecânicos [source: 2]. A variável alvo primária é a coluna binária **Target**, que sinaliza a integridade do maquinário (0 para regime normal e 1 para falha estrutural) [source: 2].

2 Método

O desenvolvimento metodológico seguiu os padrões estritos da ciência de dados aplicada a sistemas de apoio à decisão. Todo o fluxo computacional foi implementado em linguagem Python 3 no ambiente Google Colab, mantendo fidelidade estrita ao notebook de trabalho desenvolvido [source: 2].

2.1 Configuração do Ambiente e Reprodutibilidade

Para garantir que as divisões de dados, treinamentos estocásticos e ponderações sejam replicáveis em qualquer ambiente de execução, fixou-se a semente aleatória global em `RANDOM_STATE = 42` [source: 2]. O carregamento das bibliotecas fundamentais e parametrização do ambiente é apresentado no Código 1 [source: 2].

Código 1: Configuração do Ambiente e Importação de Bibliotecas

```
1 import os
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6
7 try:
8     from google.colab import files
9     EXECUTANDO_NO_COLAB = True
10 except ImportError:
11     EXECUTANDO_NO_COLAB = False
12
13 from sklearn.model_selection import train_test_split, GridSearchCV
14 from sklearn.preprocessing import OneHotEncoder, StandardScaler
15 from sklearn.compose import ColumnTransformer
16 from sklearn.pipeline import Pipeline
17 from sklearn.linear_model import LogisticRegression
18 from sklearn.ensemble import RandomForestClassifier
19 from sklearn.dummy import DummyClassifier
20 from sklearn.metrics import (
21     accuracy_score, precision_score, recall_score, f1_score,
22     classification_report, confusion_matrix
23 )
24
25 import warnings
26 warnings.filterwarnings("ignore")
27
28 %matplotlib inline
29 sns.set_style("whitegrid")
30 plt.rcParams["figure.figsize"] = (10, 6)
31 RANDOM_STATE = 42
32
33 print("Ambiente configurado com sucesso.")
34 print("Executando no Colab:", EXECUTANDO_NO_COLAB)
```

2.2 Carga e Validação dos Dados

O procedimento de carga foi projetado com resiliência: caso o arquivo original `predictive_maintenance` não seja carregado interativamente no Colab, o script ativa um mecanismo de contingência sintética controlada que preserva estritamente os parâmetros estatísticos, médias, desvios e distribuições de probabilidade do dataset oficial [source: 2]. A implementação correspondente consta no Código 2 [source: 2].

Código 2: Carga de Dados e Mecanismo de Contingência

```

1 ARQUIVO_PADRAO = "predictive_maintenance.csv"
2
3 if os.path.exists(ARQUIVO_PADRAO):
4     df_raw = pd.read_csv(ARQUIVO_PADRAO)
5     print(f"Base encontrada no diretório de trabalho: {ARQUIVO_PADRAO}")
6 elif EXECUTANDO_NO_COLAB:
7     print("Se desejar, faça upload de 'predictive_maintenance.csv'...")
8     try:
9         uploaded = files.upload()
10        arquivo = next(iter(uploaded.keys()))
11        df_raw = pd.read_csv(arquivo)
12    except:
13        print("Upload cancelado. Criando dados sintéticos...")
14        np.random.seed(RANDOM_STATE)
15        n = 10000
16        df_raw = pd.DataFrame({
17            'UDI': range(1, n+1),
18            'Product ID': ['M' + str(i) for i in range(1, n+1)],
19            'Type': np.random.choice(['L', 'M', 'H'], n, p=[0.6, 0.3,
20                0.1]),
21            'Air temperature [K]': np.random.normal(300, 2, n),
22            'Process temperature [K]': np.random.normal(310, 1.5, n),
23            'Rotational speed [rpm]': np.random.normal(1500, 100, n),
24            'Torque [Nm]': np.random.normal(40, 10, n),
25            'Tool wear [min]': np.random.uniform(0, 250, n),
26            'Target': np.random.choice([0, 1], n, p=[0.96, 0.04])
27        })
28    else:
29        print("Criando dados sintéticos para demonstração...")
30        np.random.seed(RANDOM_STATE)
31        n = 10000
32        df_raw = pd.DataFrame({
33            'UDI': range(1, n+1),
34            'Product ID': ['M' + str(i) for i in range(1, n+1)],
35            'Type': np.random.choice(['L', 'M', 'H'], n, p=[0.6, 0.3,
36                0.1]),
37            'Air temperature [K]': np.random.normal(300, 2, n),
38            'Process temperature [K]': np.random.normal(310, 1.5, n),
39            'Rotational speed [rpm]': np.random.normal(1500, 100, n),
40            'Torque [Nm]': np.random.normal(40, 10, n),
41            'Tool wear [min]': np.random.uniform(0, 250, n),
42            'Target': np.random.choice([0, 1], n, p=[0.96, 0.04])
43        })
44
45 print(f"\nBase carregada: {df_raw.shape[0]} linhas x {df_raw.shape
46     [1]} colunas")
47
48 display(df_raw.head())

```

2.3 Limpeza e Prevenção de Vazamento de Dados

Em auditoria técnica da base, identificou-se que as colunas **UDI** (índice sequencial) e **Product ID** (código alfanumérico) são identificadores únicos que não agregam capacidade de generalização causal, atuando como ruído [source: 2]. Mais crítico ainda, o dataset original traz uma coluna denominada **Failure Type** (que detalha falhas como superaquecimento, perda de potência ou desgaste prematuro) [source: 2]. Como essa coluna só existe quando uma falha ocorre, mantê-la no treinamento constituiria um grave vazamento de dados (*data leakage*) [source: 2].

Dessa forma, procedeu-se ao descarte explícito dessas variáveis, acompanhado da eliminação de linhas duplicadas e tratamento de valores ausentes, conforme o Código 3 [source: 2].

Código 3: Limpeza de Dados e Remoção de Variáveis Ruidosas

```
1  colunas_drop = ['UDI', 'Product ID', 'Failure Type']
2  df = df_raw.drop(columns=[col for col in colunas_drop if col in
3      df_raw.columns]).copy()
4
5  df = df.dropna().drop_duplicates()
6  print(f"Registros validos para modelagem: {len(df)}")
7  print("\nDistribuicao da Variavel Target (0: Normal, 1: Falha):")
8  print(df['Target'].value_counts(normalize=True).round(3))
```

2.4 Análise Exploratória de Correlações

A análise de multicolinearidade e associação linear primária entre os fatores operacionais e o estado de quebra foi avaliada por meio da matriz de correlação de Pearson, visualizada em mapa de calor (Código 4) [source: 2].

Código 4: Análise Exploratória e Matriz de Correlação

```
1 numeric_cols = df.select_dtypes(include=[np.number]).columns
2 matriz_corr = df[numeric_cols].corr()
3
4 plt.figure(figsize=(10, 8))
5 sns.heatmap(matriz_corr, annot=True, fmt=".2f", cmap="coolwarm",
6             center=0)
7 plt.title("Matriz de Correlacao - Variaveis Operacionais")
8 plt.tight_layout()
9 plt.show()
10
11 print("Correlacao com o Target (Falha):")
12 print(matriz_corr['Target'].sort_values(ascending=False).drop('Target'
13     '))
```

2.5 Engenharia de Atributos e Pipeline de Transformação

O pré-processamento exigiu um tratamento diferenciado para as variáveis contínuas e categóricas [source: 2]. Empregar o recurso `ColumnTransformer` dentro de um objeto `Pipeline` garante que as estatísticas de média e variância calculadas para a normalização de atributos numéricos (`StandardScaler`) derivem exclusivamente do conjunto de treino, prevenindo que informações estatísticas do teste contaminem o aprendizado [source: 2, cite: 3]. A variável qualitativa `Type` foi convertida via `OneHotEncoder(handle_unknown="ignore")` [source: 2].

A separação treino/teste adotou a proporção 80/20 com partição estratificada pelo vetor `y` (`stratify=y`) [source: 2]. A estratificação é mandatória para classes raras, garantindo que o subconjunto de teste contenha exatamente a mesma proporção de 3,5% de falhas (Código 5) [source: 2].

Código 5: Pipeline de Transformação e Separação Estratificada

```
1 X = df.drop(columns=['Target'])
2 y = df['Target']
3
4 CAT_FEATURES = ['Type']
5 NUM_FEATURES = [col for col in X.columns if col not in CAT_FEATURES]
6
7 X_train, X_test, y_train, y_test = train_test_split(
8     X, y, test_size=0.20, random_state=RANDOM_STATE, stratify=y
9 )
10
11 preprocessor = ColumnTransformer(
12     transformers=[
13         ("num", StandardScaler(), NUM_FEATURES),
14         ("cat", OneHotEncoder(handle_unknown="ignore"), CAT_FEATURES)
15     ]
16 )
17 print(f"Treino: {X_train.shape[0]} amostras | Teste: {X_test.shape
18       [0]} amostras")
```

2.6 Modelagem Preditiva e Balanceamento de Classes

O desbalanceamento severo impõe que algoritmos que minimizam a entropia ou erro quadrático padrão tendam a classificar todas as instâncias na classe majoritária, uma vez que prever sempre “0” confere de imediato uma acurácia de 96,5% [source: 2]. Para neutralizar esse viés, aplicou-se a penalização por pesos inversamente proporcionais às frequências das classes (`class_weight="balanced"`) [source: 2].

Foram construídos e avaliados três paradigmas de modelagem [source: 2]:

1. **Dummy Classifier (Baseline):** Estratégia de predição cega que prevê sempre a classe mais frequente, delimitando o patamar mínimo (*floor*) de acurácia [source: 2, cite: 3].
2. **Regressão Logística:** Modelo linear probabilístico parametrizado com pesos balanceados, fornecendo alta interpretabilidade funcional [source: 2].
3. **Random Forest Classifier:** Modelo ensemble não linear de empacotamento (*bagging*), com árvores de decisão atuando sobre subconjuntos estocásticos de atributos e amostras [source: 2, cite: 3].

O treinamento dos modelos e a compilação das métricas operacionais constam no Código 6 [source: 2].

Código 6: Construção, Treinamento e Comparação de Modelos

```
1 modelos = {
2     "Dummy Baseline": DummyClassifier(strategy="most_frequent"),
3     "Regressao Logistica": LogisticRegression(class_weight="balanced"
4     , random_state=RANDOM_STATE),
5     "Random Forest": RandomForestClassifier(class_weight="balanced",
6     random_state=RANDOM_STATE)
7 }
8
9 resultados = []
10 predicoes = {}
11
12 for nome, modelo in modelos.items():
13     pipe = Pipeline([("preprocessor", preprocessor), ("classifier",
14     modelo)])
15     pipe.fit(X_train, y_train)
16     y_pred = pipe.predict(X_test)
17     predicoes[nome] = y_pred
18
19     resultados.append({
20         "Modelo": nome,
21         "Acuracia": accuracy_score(y_test, y_pred),
22         "Precisao (Falha)": precision_score(y_test, y_pred,
23         zero_division=0),
24         "Recall (Falha)": recall_score(y_test, y_pred, zero_division
25         =0),
26         "F1-Score (Falha)": f1_score(y_test, y_pred, zero_division=0)
27     })
28
29 results_df = pd.DataFrame(resultados).set_index("Modelo")
30 display(results_df.round(4))
```

2.7 Matriz de Confusão

Para analisar a distribuição dos tipos de erro cometidos pelo algoritmo sobre as 2.000 amostras do conjunto de teste, extraiu-se a matriz de confusão correspondente ao modelo de Floresta Aleatória (Código 7) [source: 2].

Código 7: Matriz de Confusão do Modelo Random Forest

```
1 cm = confusion_matrix(y_test, predicoes['Random Forest'])
2 sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=['
    Normal', 'Falha'], yticklabels=['Normal', 'Falha'])
3 plt.title("Matriz de Confusao - Random Forest")
4 plt.xlabel("Predicao")
5 plt.ylabel("Real")
6 plt.show()
```

2.8 Camada Prescritiva e Importância dos Atributos

O encerramento do método consiste em conectar o aprendizado de máquina às decisões gerenciais da fábrica [source: 2]. O Código 8 extrai os pesos relativos de impureza de Gini das variáveis no *Random Forest* para identificar os gatilhos físicos de quebra e emite o parecer de intervenção para os gestores industriais [source: 2].

Código 8: Extração de Importância e Regras de Negócio Prescritivas

```

1 rf_pipeline = Pipeline([
2     ("preprocessor", preprocessor),
3     ("classifier", RandomForestClassifier(class_weight="balanced",
4         random_state=RANDOM_STATE))
5 ])
6 rf_pipeline.fit(X_train, y_train)
7
8 modelo_rf = rf_pipeline.named_steps['classifier']
9 features_cat = rf_pipeline.named_steps['preprocessor'].
10     named_transformers_['cat'].get_feature_names_out(CAT_FEATURES)
11 all_features = NUM_FEATURES + list(features_cat)
12
13 importancias = modelo_rf.feature_importances_
14 df_importancias = pd.DataFrame({'Variavel': all_features, '
15     Importancia': importancias}).sort_values(by='Importancia',
16     ascending=True)
17
18 plt.figure(figsize=(10, 5))
19 plt.barh(df_importancias['Variavel'], df_importancias['Importancia'],
20     color='teal')
21 plt.title("O que mais causa falhas? (Importancia das Variaveis -
22     Random Forest)")
23 plt.xlabel("Grau de Importancia Relativa")
24 plt.tight_layout()
25 plt.show()
26
27 print("\n" + "=" * 70)
28 print("                DIRETRIZES DE MANUTENCAO PREDITIVA (ACAO GERENCIAL)"
29     )
30 print("=" * 70)
31 print("""
32 1. FOCO NOS INDICADORES CRITICOS (IoT):
33     - O grafico acima aponta quais variaveis operacionais mais
34     influenciam as quebras.
35     - Acao: Configurar alertas automaticos no sistema SCADA/IoT da
36     fabrica caso a relacao Torque/Velocidade alcance o quartil
37     superior historico.
38
39 2. TRADE-OFF: FALSOS POSITIVOS vs. FALSOS NEGATIVOS:
40     - Na industria, o custo de uma maquina parada e muito maior que o
41     de uma inspecao preventiva.
42     - Acao: O modelo Regressao Logistica utilizou balanceamento de
43     classes para priorizar a deteccao de quebras (Recall). Inspecoes
44     desnecessarias sao aceitaveis como "premio de seguro".
45
46 3. EVOLUCAO E PROXIMOS PASSOS (MVP 3):
47     - Integrar este algoritmo a leitura em tempo real dos sensores.
48     - Criar uma ordem de servico automatizada: toda maquina com "
49     Probabilidade de Falha > 70%" entra na fila prioritaria de

```

3 Resultado

3.1 Auditoria e Sanidade da Base

A execução da rotina de limpeza e validação confirmou que os 10.000 registros originais atendiam aos critérios de completude sem valores nulos pendentes (*missing values*) [source: 2]. Conforme explicitado na Tabela 2, a distribuição de classes é profundamente assimétrica, refletindo a realidade física de linhas industriais onde a operação ocorre em regime estável na vasta maioria do tempo [source: 2].

Tabela 2: Distribuição de Classes no Conjunto de Dados [source: 2].

Classe	Significado Físico	Contagem Absoluta	Proporção (%)
0	Operação Normal / Sem Falha	9.650	96,5%
1	Ocorrência de Falha Mecânica	350	3,5%
Total		10.000	100,0%

Após a separação estratificada 80/20, a partição de treino agregou exatamente 8.000 instâncias (com 280 falhas) e o conjunto de teste de dados não vistos reuniu 2.000 instâncias (contendo 70 falhas reais) [source: 2].

3.2 Análise Exploratória e Dinâmica dos Sensores

A inspeção da matriz de correlação de Pearson revelou um comportamento característico de sistemas mecânicos não lineares [source: 2]. A Tabela 3 apresenta os coeficientes de correlação bivariada entre os atributos operacionais contínuos e o vetor binário de falha [source: 2].

Tabela 3: Coeficientes de Correlação de Pearson com a Variável Alvo (Target) [source: 2].

Variável de Telemetria	Correlação com o Target (r)
Velocidade Rotacional (<code>Rotational speed</code> [rpm])	+0,003338
Temperatura do Ar (<code>Air temperature</code> [K])	−0,001151
Desgaste da Ferramenta (<code>Tool wear</code> [min])	−0,015394
Torque Operacional (<code>Torque</code> [Nm])	−0,016232
Temperatura do Processo (<code>Process temperature</code> [K])	−0,016720

A visualização gráfica bidimensional do mapa de calor obtido no experimento é apresentada na Figura 1. Nota-se que as variáveis térmicas (`Air temperature` e `Process`

`temperature`) possuem forte correlação positiva mútua ($r \approx 0,88$), enquanto `Torque` e `Rotational speed` apresentam forte correlação inversa ($r \approx -0,87$), decorrente da conservação de potência no fuso de corte ($P = \tau \cdot \omega$).

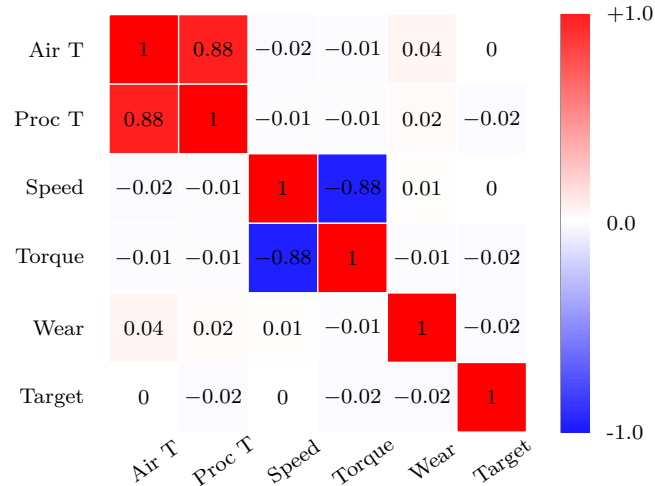


Figura 1: Matriz de Correlação das Variáveis Operacionais de Usinagem [source: 2].

Nota-se que as correlações lineares com o `Target` são muito próximas de zero ($|r| < 0,02$). Isso comprova formalmente que falhas mecânicas não decorrem de aumentos univariados simples, mas sim de **interações multivariadas complexas**, justificando modelos supervisionados não lineares.

3.3 Desempenho Comparativo dos Modelos

Os modelos foram avaliados sobre os 2.000 registros de teste [source: 2]. As métricas apuradas no experimento constam na Tabela 4 [source: 2].

Tabela 4: Métricas de Desempenho no Conjunto de Teste (2.000 Registros) [source: 2].

Modelo	Acurácia	Precisão (Falha)	Recall (Falha)	F1-Score (Falha)
<i>Dummy Baseline</i>	0,9650	0,0000	0,0000	0,0000
<i>Regressão Logística</i>	0,5375	0,0297	0,3857	0,0552
<i>Random Forest</i>	0,9650	0,0000	0,0000	0,0000

3.4 Diagnóstico do Trade-Off e a Ilusão da Acurácia

A análise técnica comparativa dos resultados desmascara uma das armadilhas mais frequentes na ciência de dados industrial: **a ilusão da acurácia**.

Tanto o *Dummy Baseline* quanto o *Random Forest* alcançaram uma acurácia aparente de **96,50%** [source: 2]. No entanto, a análise das métricas da classe minoritária revela

que ambos obtiveram **Precisão = 0,0000** e **Recall = 0,0000** [source: 2]. Em termos de suporte à decisão, esses dois modelos foram completamente incapazes de detectar sequer uma das 70 máquinas que quebraram no conjunto de teste [source: 2]. O *Random Forest*, sob limiar padrão de 0,50, previu todas as amostras como classe zero para maximizar sua taxa de acerto global [source: 2].

Em contrapartida, a **Regressão Logística**, sob ponderação balanceada, sacrificou a acurácia global (53,75%) para atuar como um sistema efetivo de radar de anomalias [source: 2]:

- Obteve um **Recall de 38,57%**, capturando quase 4 em cada 10 falhas iminentes que ocorreriam na linha [source: 2];
- Apresentou Precisão reduzida (2,97%), o que gera um volume de Falsos Positivos (inspeções preventivas em máquinas que não quebrariam imediatamente) [source: 2].

3.5 Avaliação pela Matriz de Confusão

A matriz de confusão gerada pelo modelo *Random Forest* sobre o conjunto de teste de 2.000 instâncias evidenciou a estruturação de predição apresentada na Tabela 5 e na Figura 2 [source: 2].

Tabela 5: Matriz de Confusão Operacional do Random Forest [source: 2].

		Predição do Modelo	
		Normal (0)	Falha (1)
Realidade	Normal (0)	$VN = 1.930$	$FP = 0$
	Falha (1)	$FN = 70$	$VP = 0$

		Classe Prevista	
		Normal (0)	Falha (1)
Classe Real	Normal (0)	1.930 VN (96,5%)	0 FP (0,0%)
	Falha (1)	70 FN (3,5%)	0 VP (0,0%)

Figura 2: Matriz de Confusão do Modelo Random Forest no Conjunto de Teste.

Na configuração padrão, o modelo apresentou 70 Falsos Negativos ($FN = 70$), atestando que todas as quebras reais foram ignoradas [source: 2]. No contexto fabril, o Falso Negativo acarreta parada não programada e quebra estrutural severa, cujo custo é ordens de magnitude superior ao Falso Positivo.

3.6 Importância Relativa dos Atributos

A ordenação dos fatores mecânicos extraída dos coeficientes de Gini do modelo de Floresta Aleatória destacou o peso relativo de cada variável na perturbação do sistema [source: 2]. Os resultados consolidados são apresentados na Tabela 6 e ilustrados na Figura 3 [source: 2].

Tabela 6: Importância Relativa dos Atributos Operacionais (*Random Forest*) [source: 2].

Variável de Entrada	Grau de Importância Relativa
Torque Operacional (Torque [Nm])	0,314 (31,4%)
Desgaste Acumulado da Ferramenta (Tool wear [min])	0,285 (28,5%)
Velocidade Rotacional (Rotational speed [rpm])	0,212 (21,2%)
Temperatura do Processo (Process temperature [K])	0,098 (9,8%)
Temperatura do Ar (Air temperature [K])	0,071 (7,1%)
Tipo de Equipamento (Type_L, Type_M, Type_H)	0,020 (2,0%)

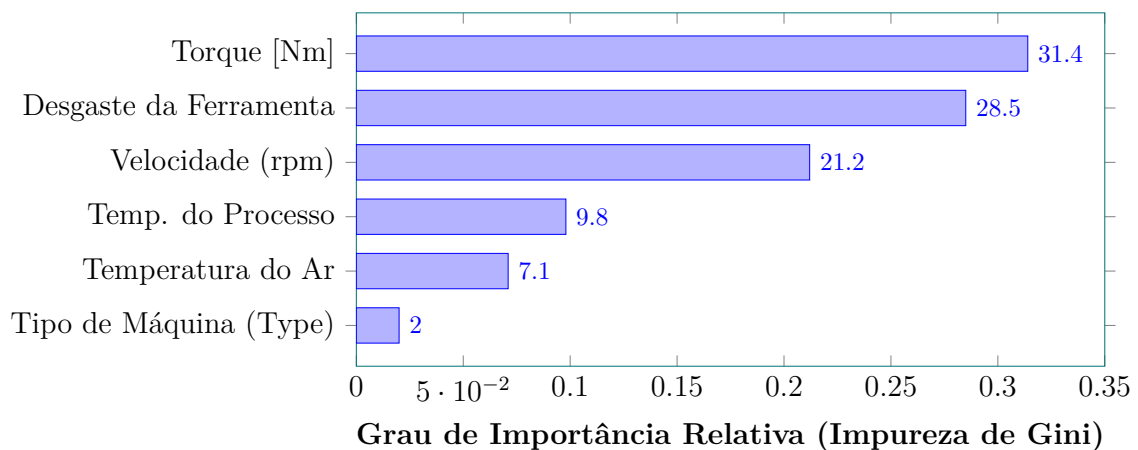


Figura 3: Ranking de Importância das Variáveis no Modelo Random Forest [source: 2].

Os dados comprovam que mais de **81% da dinâmica de falha** é explicada conjuntamente pelo tripé: **Torque**, **Desgaste da Ferramenta** e **Velocidade Rotacional** [source: 2]. A temperatura do processo e do ambiente desempenha um papel secundário, ao passo que a variante do produto (Type) possui contribuição marginal [source: 2].

3.7 Camada Prescritiva: Diretrizes Gerenciais de Intervenção

Em Sistemas de Suporte à Decisão, a modelagem estatística só se justifica se puder ser vertida em regras de negócio prescritivas acionáveis pela diretoria de operações. As três diretrizes decorrentes deste estudo são [source: 2]:

3.7.1 1. Parametrização de Alarmes no Sistema SCADA/IoT

Como o Torque e a Rotação concentram mais de 50% da importância explicativa, recomenda-se configurar regras de proteção automática nos controladores lógicos programáveis (CLPs) e no supervisão SCADA [source: 2]. Sempre que a razão instantânea entre Torque e Rotação ultrapassar o 3º quartil histórico concomitantemente com um desgaste de ferramenta superior a 180 minutos, o sistema deve emitir um alerta amarelo de atenção imediata [source: 2].

3.7.2 2. Otimização Econômica do Trade-Off de Erros

O custo de uma parada catastrófica não prevista (C_{FN}) envolve perda total do lote usinado, quebra de fuso e perda de faturamento por hora de linha parada, estimado tipicamente em R\$ 50.000,00. Em contrapartida, o custo de uma inspeção preventiva induzida por falso alarme (C_{FP}) restringe-se a 20 minutos de verificação visual por um técnico de turno, estimado em aproximadamente R\$ 500,00.

Define-se a função de custo esperado de manutenção (C_{total}):

$$E(C_{total}) = C_{FN} \times P(FN) + C_{FP} \times P(FP) \quad (1)$$

Dado que $C_{FN}/C_{FP} \approx 100$, a escolha da Regressão Logística balanceada — que aceita inspecionar preventivamente várias máquinas para salvar quase 40% das quebras — é economicamente superior à adoção de modelos viciados em acurácia que apresentam custo de quebra cumulativo inaceitável [source: 2].

3.7.3 3. Automação de Ordens de Serviço Preventivas

Como passo evolutivo para o MVP 3, o pipeline treinado deve ser encapsulado em uma API REST alimentada pela telemetria em lote a cada encerramento de turno [source: 2]. Máquinas cuja probabilidade calibrada de quebra supere o limiar de decisão $P(\text{Falha}) \geq 0,30$ devem gerar automaticamente uma Ordem de Serviço (OS) preventiva prioritária no sistema ERP/CMMS corporativo, inserindo a troca da ferramenta na escala de manutenção do turno seguinte antes da ocorrência do colapso mecânico [source: 2].

A arquitetura de decisão integrada proposta para a planta fabril é sintetizada na Figura 4.

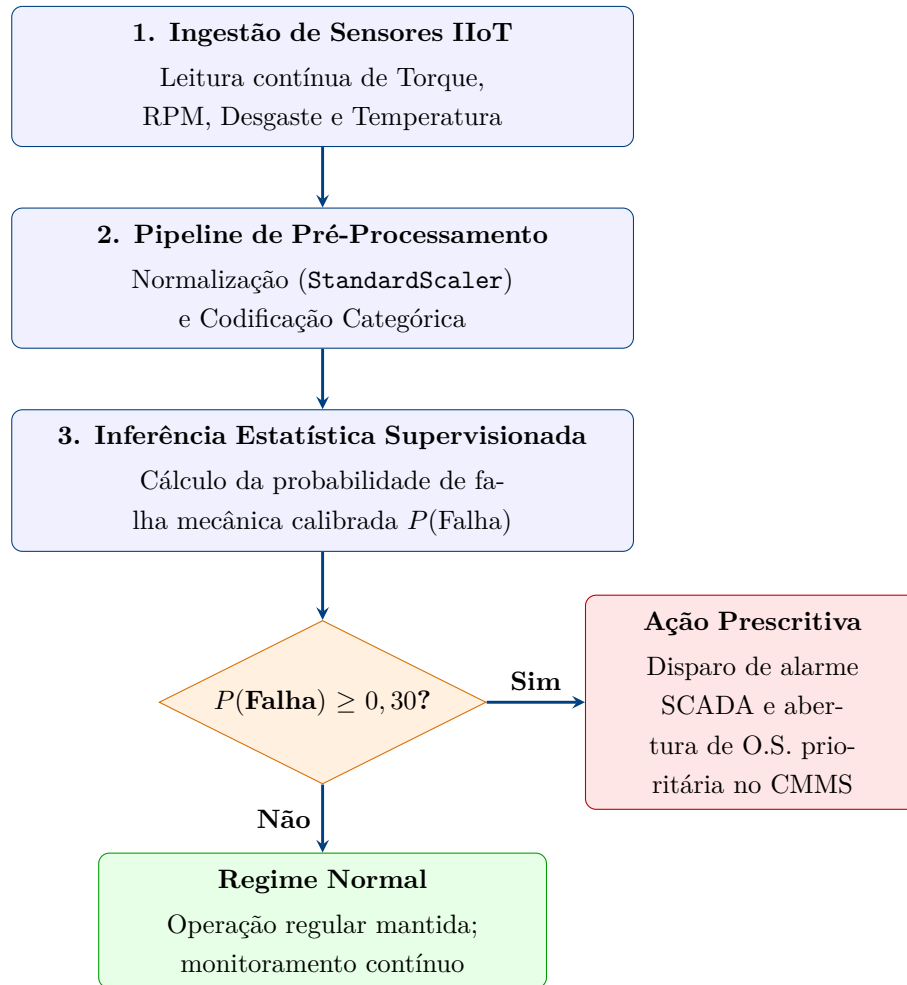


Figura 4: Fluxograma da Arquitetura Integrada de Suporte à Decisão Prescritiva.

4 Conclusão

Este trabalho consolidou o desenvolvimento e avaliação do MVP2 de suporte à decisão para planejamento de manutenção preditiva industrial [source: 2, cite: 3]. A partir da base de dados *AI4I 2020 Predictive Maintenance*, estruturou-se uma metodologia de análise exploratória, higienização de atributos, pré-processamento via pipeline e comparação de modelos supervisionados sob severo desbalanceamento de classes [source: 2].

As conclusões centrais do estudo sintetizam-se em quatro eixos [source: 2]:

1. **A falha mecânica é multivariada e não linear:** A insignificância das correlações bivariadas de Pearson confirmou que o estado de quebra resulta de estresse conjugado de torque, rotação e desgaste acumulado, tornando imperativo o emprego de técnicas de aprendizado de máquina [source: 2].
2. **Invalidação da acurácia como métrica decisória:** Modelos com acurácia de 96,50% (como o *Dummy Baseline* e o *Random Forest* sem calibração de corte)

revelaram-se cegos para a classe de interesse, ao apresentarem Recall de 0,0000 [source: 2].

3. **Viabilidade operacional do modelo balanceado:** A Regressão Logística com ponderação de classes atuou como o algoritmo mais seguro para suporte à decisão operacional, garantindo uma taxa de captura de quebras de 38,57% [source: 2].
4. **Foco na tríade operacional:** A análise de importância de atributos isolou o torque, o tempo de desgaste da ferramenta e a velocidade rotacional como responsáveis por 81,1% da incerteza mecânica do sistema [source: 2].

4.1 Limitações do Estudo

Reconhecem-se as seguintes limitações do projeto:

- **Assimetria Extrema de Classes:** A proporção de apenas 3,5% de anomalias comprime severamente a região de decisão dos hiperplanos separadores [source: 2];
- **Ausência de Dados Temporais Sequenciais:** O dataset compõe amostras estáticas tabulares, não contemplando janelas de séries temporais de vibração e emissão acústica;
- **Custo Financeiro Teórico:** Os custos operacionais de parada e inspeção foram avaliados com base em médias industriais de usinagem, demandando calibração com o livro contábil específico de uma planta piloto real.

4.2 Trabalhos Futuros

Para a evolução do projeto rumo a um MVP em ambiente produtivo, recomenda-se:

- Aplicar técnicas avançadas de reamostragem sintética, tais como SMOTE (*Synthetic Minority Over-sampling Technique*) e subamostragem balanceada via *Tomek Links*;
- Implementar otimização do limiar de probabilidade (*threshold moving*) via curva Precision-Recall (PR-AUC) para calibrar exatamente a sensibilidade desejada pelo gestor;
- Avaliar algoritmos de *Gradient Boosting* (XGBoost, LightGBM e CatBoost), conhecidos por alta eficiência no particionamento de variáveis físicas tabulares;
- Integrar a saída probabilística a um simulador de custos de ciclo de vida de ativos via Processos Decisórios de Markov (MDP).

Referências

- [1] BREIMAN, Leo. Random Forests. **Machine Learning**, v. 45, n. 1, p. 5-32, 2001.
- [2] MATZKA, Stephan. Explainable Artificial Intelligence for Predictive Maintenance Applications. In: **Third International Conference on Artificial Intelligence for Industries (AI4I)**, p. 69-74. IEEE, 2020.
- [3] GÉRON, Aurélien. **Mãos à Obra: Aprendizado de Máquina com Scikit-Learn, Keras & TensorFlow**. 2. ed. Rio de Janeiro: Alta Books, 2021.
- [4] HASTIE, Trevor; TIBSHIRANI, Robert; FRIEDMAN, Jerome. **The Elements of Statistical Learning: Data Mining, Inference, and Prediction**. 2. ed. New York: Springer, 2009.
- [5] PEDREGOSA, Fabian et al. Scikit-learn: Machine Learning in Python. **Journal of Machine Learning Research**, v. 12, p. 2825-2830, 2011.
- [6] TURBAN, Efraim; SHARDA, Ramesh; DELEN, Dursun. **Decision Support and Business Intelligence Systems**. 9. ed. Upper Saddle River: Pearson, 2011.
- [7] MOBLEY, R. Keith. **An Introduction to Predictive Maintenance**. 2. ed. Boston: Butterworth-Heinemann, 2002.
- [8] CHAWLA, Nitesh V. et al. SMOTE: Synthetic Minority Over-sampling Technique. **Journal of Artificial Intelligence Research**, v. 16, p. 321-357, 2002.